

Lab - Creating a Custom MySQL Image

To Begin with the Lab

Summary

This lab demonstrates how to create a **custom MySQL Docker image** that automatically initializes the database during container startup. Instead of manually creating the database, users, tables, and sample data after launching the container, all these tasks are executed automatically using a SQL initialization script included in the Docker image.

Understanding

The official MySQL Docker image allows automatic database initialization by executing any SQL files placed inside the **/docker-entrypoint-initdb.d** directory. When the MySQL container starts for the first time, it runs all SQL scripts in this directory.

By creating a **custom Docker image**, you can preconfigure:

- Database creation
- User creation
- Granting permissions
- Table creation
- Sample data insertion

As a result, every new container created from this image is fully initialized and ready to use without any manual SQL commands.

Example:

Instead of manually running:

- Create Database
- Create User
- Grant Privileges
- Create Table
- Insert Data

the container performs all these steps automatically during startup.

Lab Steps

Step 1: Create a Project Folder

- Create an empty folder for the **MySQL Docker** project.
- Store all required files inside this folder.

Example:

MySQL-Docker/

```
|
|— Dockerfile
|— init.sql
```

Step 2: Create the SQL Initialization Script

- Create a file named: `init.sql`
- Paste the code in `init.sql`.

```
-- init.sql (MySQL)

-- Create database
CREATE DATABASE IF NOT EXISTS cloudxeusdb;
USE cloudxeusdb;

-- Create app user (lab-friendly)
CREATE USER IF NOT EXISTS 'appuser'@'%' IDENTIFIED BY 'StrongPassword!123';
```

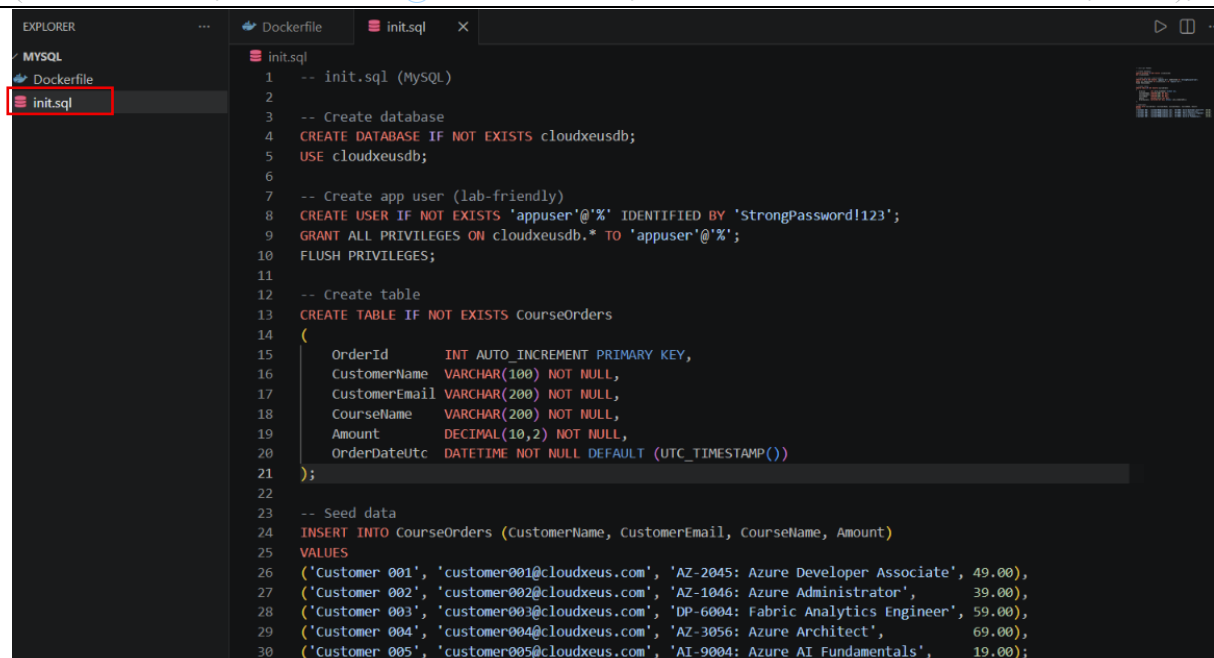
```
GRANT ALL PRIVILEGES ON cloudxeusdb.* TO 'appuser'@'%';
FLUSH PRIVILEGES;
```

-- Create table

```
CREATE TABLE IF NOT EXISTS CourseOrders
(
  OrderId      INT AUTO_INCREMENT PRIMARY KEY,
  CustomerName VARCHAR(100) NOT NULL,
  CustomerEmail VARCHAR(200) NOT NULL,
  CourseName   VARCHAR(200) NOT NULL,
  Amount       DECIMAL(10,2) NOT NULL,
  OrderDateUtc DATETIME NOT NULL DEFAULT (UTC_TIMESTAMP())
);
```

-- Seed data

```
INSERT INTO CourseOrders (CustomerName, CustomerEmail, CourseName, Amount)
VALUES
('Customer 001', 'customer001@cloudxeus.com', 'AZ-2045: Azure Developer Associate', 49.00),
('Customer 002', 'customer002@cloudxeus.com', 'AZ-1046: Azure Administrator', 39.00),
('Customer 003', 'customer003@cloudxeus.com', 'DP-6004: Fabric Analytics Engineer', 59.00),
('Customer 004', 'customer004@cloudxeus.com', 'AZ-3056: Azure Architect', 69.00),
('Customer 005', 'customer005@cloudxeus.com', 'AI-9004: Azure AI Fundamentals', 19.00);
```



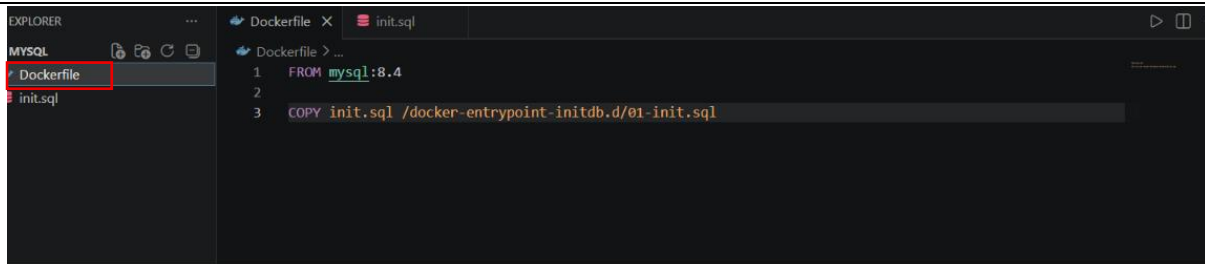
```
1  -- init.sql (MySQL)
2
3  -- Create database
4  CREATE DATABASE IF NOT EXISTS cloudxeusdb;
5  USE cloudxeusdb;
6
7  -- Create app user (lab-friendly)
8  CREATE USER IF NOT EXISTS 'appuser'@'%' IDENTIFIED BY 'StrongPassword!123';
9  GRANT ALL PRIVILEGES ON cloudxeusdb.* TO 'appuser'@'%' ;
10 FLUSH PRIVILEGES;
11
12 -- Create table
13 CREATE TABLE IF NOT EXISTS CourseOrders
14 (
15   OrderId      INT AUTO_INCREMENT PRIMARY KEY,
16   CustomerName VARCHAR(100) NOT NULL,
17   CustomerEmail VARCHAR(200) NOT NULL,
18   CourseName   VARCHAR(200) NOT NULL,
19   Amount       DECIMAL(10,2) NOT NULL,
20   OrderDateUtc DATETIME NOT NULL DEFAULT (UTC_TIMESTAMP())
21 );
22
23 -- Seed data
24 INSERT INTO CourseOrders (CustomerName, CustomerEmail, CourseName, Amount)
25 VALUES
26 ('Customer 001', 'customer001@cloudxeus.com', 'AZ-2045: Azure Developer Associate', 49.00),
27 ('Customer 002', 'customer002@cloudxeus.com', 'AZ-1046: Azure Administrator', 39.00),
28 ('Customer 003', 'customer003@cloudxeus.com', 'DP-6004: Fabric Analytics Engineer', 59.00),
29 ('Customer 004', 'customer004@cloudxeus.com', 'AZ-3056: Azure Architect', 69.00),
30 ('Customer 005', 'customer005@cloudxeus.com', 'AI-9004: Azure AI Fundamentals', 19.00);
```

Step 3: Create the Dockerfile

- Create a file named: Dockerfile
- Paste the code in **Dockerfile**

```
FROM mysql:8.4
```

COPY init.sql /docker-entrypoint-initdb.d/01-init.sql



The screenshot shows the Visual Studio Code interface. On the left, the Explorer pane shows a file named 'init.sql' under a folder named 'MYSQL'. The main editor area shows the 'Dockerfile' with the following content:

```
1 FROM mysql:8.4
2
3 COPY init.sql /docker-entrypoint-initdb.d/01-init.sql
```

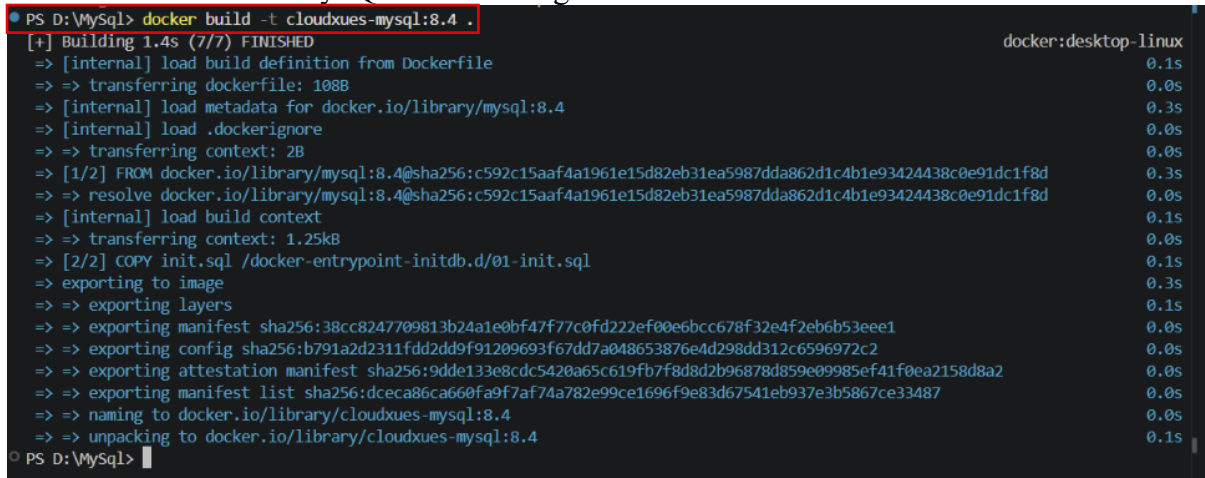
Step 4: Build the Custom Docker Image

Open the terminal inside the project folder.

Run:

docker build -t cloudues-mysql:8.4 .

This creates a custom MySQL Docker image.



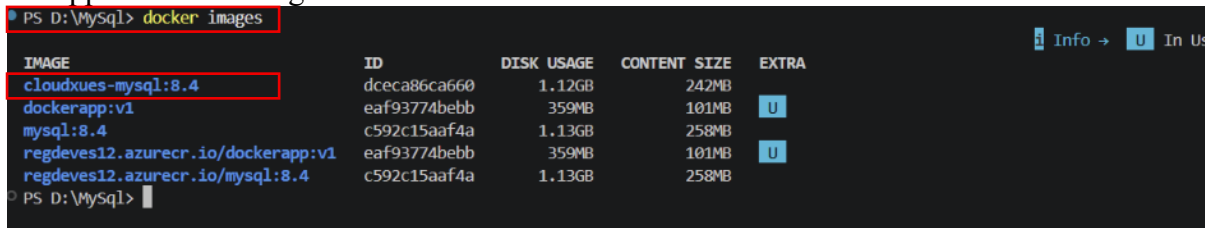
The screenshot shows a terminal window with the command `PS D:\MySQL> docker build -t cloudues-mysql:8.4 .` and its output. The output shows the build process for the Docker image, including loading the build definition, transferring the Dockerfile, loading metadata for the base image (mysql:8.4), and copying the init.sql file into the image. The final output is `PS D:\MySQL>`.

Step 5: Verify the Image

• Run:

docker images

- Verify that:
- Cloudxeus-mysql:8.4
- appears in the image list.

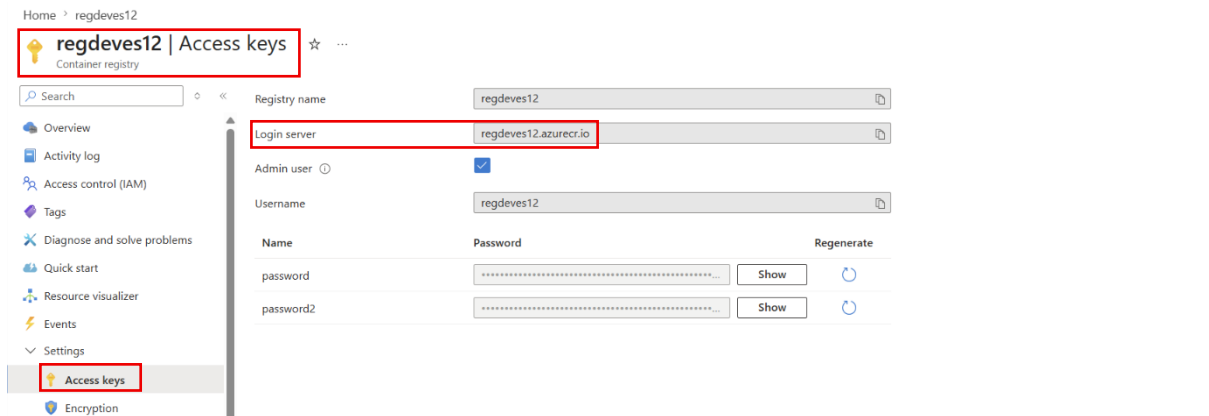


The screenshot shows a terminal window with the command `PS D:\MySQL> docker images` and its output. The output is a table listing the Docker images:

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
cloudues-mysql:8.4	dceca86ca660	1.12GB	242MB	
dockerapp:v1	eaf93774bebb	359MB	101MB	U
mysql:8.4	c592c15aaf4a	1.13GB	258MB	
regdeves12.azurecr.io/dockerapp:v1	eaf93774bebb	359MB	101MB	U
regdeves12.azurecr.io/mysql:8.4	c592c15aaf4a	1.13GB	258MB	

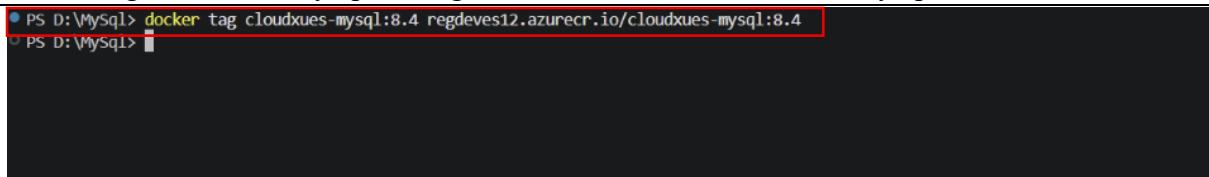
Step 6: Tag the Image for Azure Container Registry

- Go to the Azure container Registry.
- Navigate to Access keys.
- Copy the Login server.



Run:

```
docker tag cloudxues-mysql:8.4 regdevest12.azurecr.io/cloudxues-mysql:8.4
```

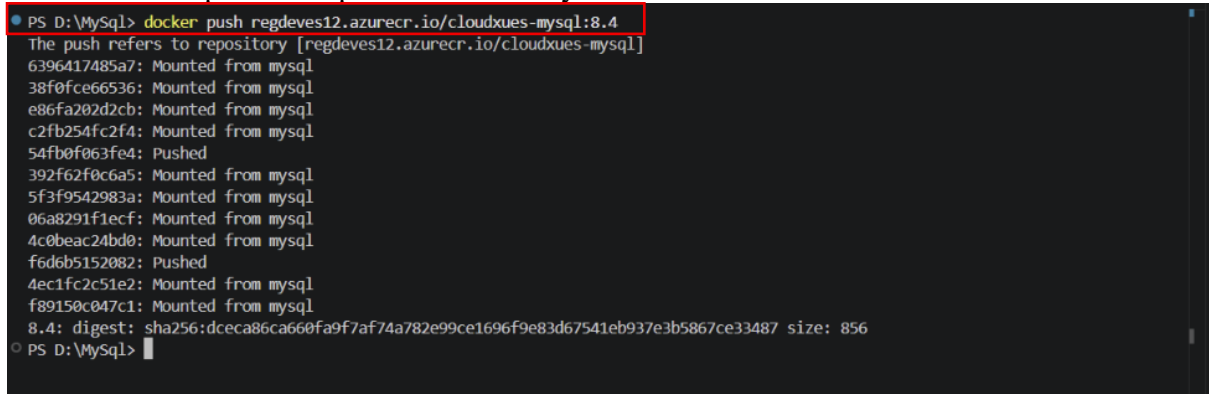


Step 7: Push the Image to Azure Container Registry

Run:

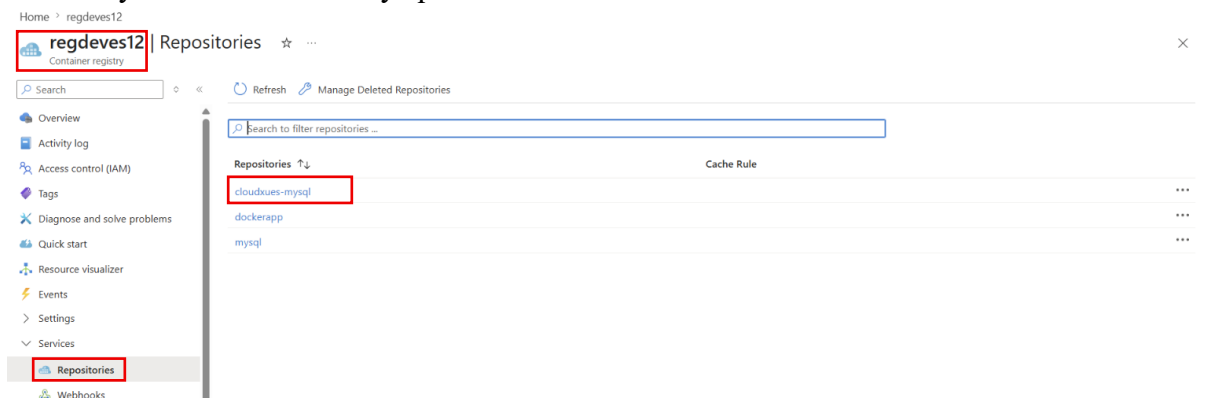
```
docker push regdevest12.azurecr.io/Cloudxues-mysql:8.4
```

Wait until the upload completes successfully.



Step 8: Verify the Image in Azure Container Registry

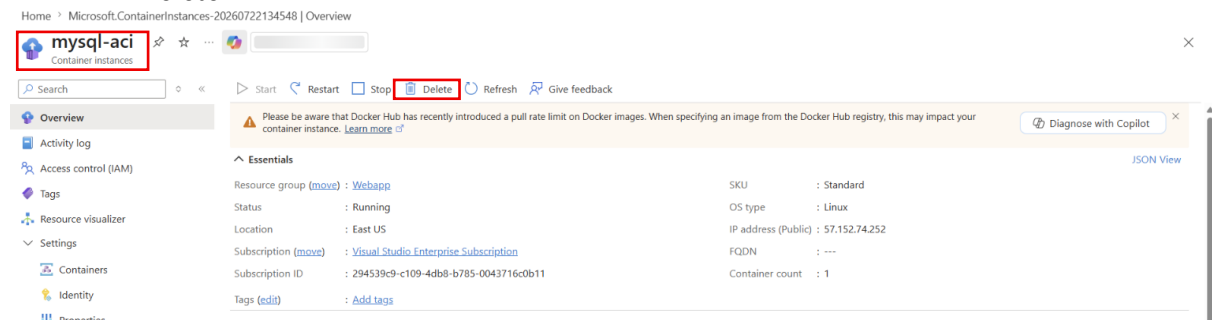
- Open Azure Container Registry.
- Select **Repositories**.
- Verify that: Cloudxues-mysql:8.4 is available.



Step 9: Delete the Existing MySQL Container Instance

- Open Azure Portal.

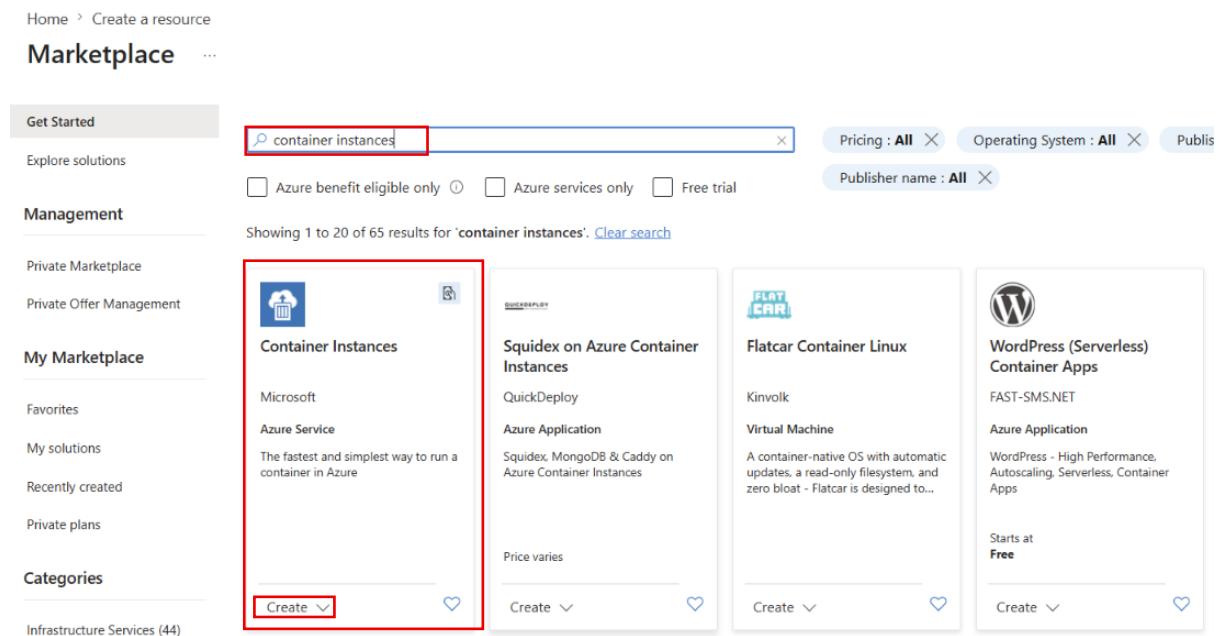
- Navigate to the existing **Container Instance**.
- Click **Delete**.



- Confirm the deletion.

Step 10: Create a New Azure Container Instance

- Click **Create Resource**.
- Search for **Container Instances**.
- Click **Create**.



Step 11: Configure Basic Settings

- Select the existing **Resource Group**.
- Enter a **Container Name**.
- Select the **Region** (Example: East US).

Create container instance ...

Basics Networking Monitoring Advanced Tags Review + create

Azure Container Instances (ACI) allows you to quickly and easily run containers on Azure without managing servers or having to learn new tools. ACI offers per-second billing to minimize the cost of running containers on the cloud.

[Learn more about Azure Container Instances](#)

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ⓘ Visual Studio Enterprise Subscription

Resource group * ⓘ Webapp

[Create new](#)

Container details

Container name * ⓘ mysql-aci

Region * ⓘ (US) East US

Availability zones (Preview) ⓘ None

SKU Standard

Step 12: Select the Custom Image

- Set **Image Source** to **Azure Container Registry**.
- Select your **Azure Container Registry**.
- Choose the repository: **cloudxues-mysql**.
- Select Image tag **8.4**

Create container instance ...

Region * ⓘ (US) East US

Availability zones (Preview) ⓘ None

SKU Standard

Image source * ⓘ

☐ Quickstart images

☒ Azure Container Registry

☐ Other registry

Run with Azure Spot discount ⓘ

☐ Spot containers are not available in the selected region. [Learn more](#)

Registry * ⓘ regdeves12

i If you do not see your Azure Container Registry, ensure you have been assigned the Reader Role for the Azure Container Registry or select an Azure Container Registry in a different subscription. [Learn more](#)

Image * ⓘ cloudxues-mysql

Image tag * ⓘ 8.4

OS type * ☒ Linux ☐ Windows

Size * ⓘ 1 vcpu, 1.5 GiB memory, 0 gpus

[Change size](#)

Step 13: Configure Networking

- Enable **Public IP** (for the lab).
- Expose **TCP Port 3306**.

Create container instance ...

Basics Networking Monitoring Advanced Tags Review + create

Choose between three networking options for your container instance:

- **'Public'** will create a public IP address for your container instance.
- **'Private'** will allow you to choose a new or existing virtual network for your container instance.
- **'None'** will not create either a public IP or virtual network. You will still be able to access your container logs using the command line.

Networking type ☒ Public ☐ Private ☐ None

DNS name label ⓘ

DNS name label scope reuse ⓘ Any reuse (unsecure) ▼

Ports ⓘ

Ports	Ports protocol
3306 ✓	TCP ▼

Step 14: Configure Environment Variables

Add the required MySQL environment variable:

Variable	Value
MYSQL_ROOT_PASSWORD	mysql@123

Mark the password as **Secure**.

Create container instance ...

Basics Networking Monitoring Advanced Tags Review + create

Configure additional container properties and variables.

Restart policy ⓘ On failure ▼

Environment variables

Mark as secure

	Key	Value
No ▼	MYSQL_ROOT_PASSWORD ✓	mysql@123 ✓
No ▼		

Command override ⓘ Example: ["/bin/bash", "-c", "echo hello; sleep 100000"]

Key management ⓘ ☒ Microsoft-managed keys (MMK) ☐ Customer-managed keys (CMK)

Step 15: Review and Deploy

- Click **Review + Create**.
- Validate the configuration.
- Click **Create**.

Validation passed

BasicsNetworkingMonitoringAdvancedTagsReview + create

Basics

Subscription

Visual Studio Enterprise Subscription

Resource group

Webapp

Region

East US

Container name

mysql-aci

SKU

Standard

Image type

Private

Image registry login server

regdevs12.azurecr.io

Image

regdevs12.azurecr.io/cloudxues-mysql:8.4

Image registry user name

regdevs12

OS type

Linux

Memory (GiB)

1.5

Number of CPU cores

1

GPU type (preview)

None

GPU count

0

Networking

Networking type

Public

Create< PreviousNext >Download a template for automation

- Wait until deployment completes.

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

Home

Microsoft.ContainerInstances-20260722154911 | Overview

Deployment

Search

DeleteCancelRedeployDownloadRefresh

Overview

Inputs

Outputs

Template

✓ Your deployment is complete

Deployment name : Microsoft.ContainerInstances-20260722154911

Subscription : Visual Studio Enterprise Subscription

Resource group : Webapp

Start time : 7/22/2026, 4:03:23 PM

Correlation ID : 86ebfaf9-15e9-431a-9898-023257ce48ff

> Deployment details

< Next steps

Go to resource

- ## Step 16: Verify the Container
- Open the deployed Container Instance.
 - Navigate to Containers.
 - Verify the container state is: Running

Home > Microsoft.ContainerInstances-20260722154911 | Overview > mysql-aci

mysql-aci | Containers

Search

RefreshGive feedback

Overview

Activity log

Access control (IAM)

Tags

Resource visualizer

Settings

Containers

1 container and 0 init containers

Name	Image	State	Previous state	Start time	Restart count
mysql-aci	regdevs12.azurecr.io/cloudxues-mys...	Running	-	2026-07-22T10:34:18.48Z	0